Intermediate (build real memory) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the classic bug
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the classic bug

The whole exercise is one idea: a Python list, mutated in place, that grows by exactly two entries per turn — one `{"role": "user", ...}` and one `{"role": "assistant", ...}`. Every single call sends this entire list, not just the newest message; the model has no server-side memory of its own (this is the exact "No memory" idea from Doc03).

The response object doesn't give you back a ready-made message dict — you build one yourself from the reply text: `{"role": "assistant", "content": response.choices[0].message.content}`. Append this to `history`, not just print it, or the next turn's call won't include it.

Also decide how the loop ends — a sentinel input like typing `"quit"` is the simplest approach: check for it right after reading input, before appending anything, so the exit command itself never becomes part of the conversation history sent to the model.

Hint 2 — The plan, and almost the whole thing

```
function get_reply(history) -> str:
    call the API with messages=history
    return the reply text

function run_chat():
    history = [system message]
    loop:
        read user_input; if "quit": stop
        history.append(user message)
        reply = get_reply(history)
```

```
        history.append(assistant message)
        print reply
```

```
# conversation_memory_practice.py
def get_reply(history: list) -> str:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
    )
    return response.choices[0].message.content

def run_chat() -> None:
    history = [
        {"role": "system", "content": "You are a helpful assistant."}
    ]

    while True:
        user_input = input("You: ")
        if user_input == "quit":
            break

        history.append({"role": "user", "content": user_input})
        reply = get_reply(history)
        history.append({"role": "assistant", "content": reply})

        print("Assistant:", reply)
```

Trace through 2-3 turns by hand and confirm `history` has the length you expect after each one, then write the `client` /import lines yourself, then compare against the Solution.

Full solution: Show me the solution